FULL-PROCESS AUDIT & IMPLEMENTATION REPORT

AquaLog

Shrimp Farm Management Application

A complete engineering audit of the offline-first Flutter application — eleven critical defects located and fixed, data-integrity flows rebuilt on real records, and every change verified by static analysis and the full 164-case regression suite.

Z.ai Engineering Audit

PRODUCT · UX · ENGINEERING REVIEW

SEPTEMBER 2026 · BASELINE V2.0 · FLUTTER 3.47.3

Table of Contents

1. Project Understanding 2

2. Existing Problems: Audit Findings 3

2.1 P1 and P2 findings 4

3. UI/UX Improvements 4

4. Layout & Workflow Improvements 5

5. Functional & Data-Integrity Improvements 5

5.1 Backup and restore, version 2 6

5.2 Real CSV exports and wired intelligence 6

6. Design System 7

7. Technical & Code-Quality Improvements 8

8. Changed Files 8

9. Remaining Issues 10

10. Future Recommendations 10

1. Project Understanding

AquaLog is an offline-first farm management application for intensive Pacific white shrimp (*Litopenaeus vannamei*) cultivation, built for Indian pond operators who frequently work without reliable connectivity. The application tracks the complete production cycle: sites and ponds are registered first, batches of post-larvae are stocked into ponds, and every subsequent activity—feed sessions, cast-net samplings, water-quality logs, hapa survival checks, mortality events, treatments, and machine runs—is recorded against the owning batch. Financial modules track expenses, revenues, market prices, and inventory, while intelligence features such as the daily feed plan, the harvest decision engine, and the pond health score turn raw records into actionable guidance.

The codebase is a single Flutter application targeting Android primarily, with desktop and web shells present. State management uses Riverpod providers over a Drift (SQLite) database that is generated from 23 user-data tables. The UI follows Material 3 with a bilingual English/Hindi string catalog routed through a settings-level translation helper. Navigation is organized around seven bottom tabs: Overview, Manage, Stock, Log Feed, Finance, Reports, and Settings, with a global search dialog reaching into every feature area.

90
Dart source files (non-generated)

23
User-data tables in Drift schema

21
Test suites — 164 test cases

7
Primary navigation tabs

Attribute	Detail
Product domain	Offline-first shrimp (L. vannamei) farm management for Indian pond operators
Framework	Flutter 3.47.3 / Dart 3.13.3, Material 3 design language
Persistence	Drift ORM over SQLite; 23 user-data tables; local-only, no server sync
State management	Riverpod 2.6 (providers for DB access, settings, dashboard contexts)
Localization	EN/HI string catalog (AppStrings) consumed via settings.tr(key) helper
Data hierarchy	FarmSite → Pond → Batch, with per-batch feed/sampling/WQ ledgers
Intelligence layer	Daily feed plan, check-tray engine, harvest decision engine, health score
Quality gates	flutter analyze + 21 test files covering DB, engines, and regressions

Table 1 — Application profile at the start of this engagement.

The engagement brief followed a strict improve-over-rebuild philosophy: understand the entire project before changing it, preserve existing business behaviour and data, and prioritise fatal defects before polish. All findings below were verified directly against the source, and every implemented change was re-verified with static analysis and the full regression suite rather than by inspection alone.

2. Existing Problems: Audit Findings

The audit surfaced eleven P0 defects—issues that corrupt data, crash the app, or present fabricated numbers as fact. Each was reproduced by reading the exact code paths involved before any fix was written. The most consequential cluster concerns data trustworthiness: the harvest completion dialog wrote hard-coded metrics (FCR 1.30, survival 88%, count derived as kg×40) into the permanent record regardless of the batch's real feed ledger, and the backup service exported ten tables but restored only five while wiping fifteen, guaranteeing silent data loss on every restore.

#	P0 Finding	Impact
1	Harvest dialog writes fabricated FCR / survival / count constants	Permanent record falsified; downstream analytics meaningless
2	Backup v1 restores 5 of 10 exported tables, wipes 15	Guaranteed silent data loss on every restore
3	Water-source enum mismatch between create and edit dialogs	Dropdown crash when editing sites created by the other flow
4	batchNumber hard-coded to 1 in the pond-scoped stocking flow	Duplicate batch numbers; per-pond history unusable
5	CSV export builds a string in memory, never writes a file	User told “exported”; nothing leaves the device
6	Check-tray buttons mislabelled vs. engine zone thresholds	Feed adjustment engine receives the wrong zone
7	Factory-reset table list drifted from backup-restore list	Reset leaves orphan rows; restore drops data
8	feed_logging_title translation key missing from catalog	Raw key rendered to end users
9	~500 lines of duplicated dialogs diverged across two pages	Same dialog behaves differently per entry point
10	855 lines of inventory detail pages unreachable	Dead code; inventory drill-down impossible
11	DailyFeedPlanCard intelligence inputs never wired	Feed plan ignores tray, DO, ammonia, weather, mortality

Table 2 — P0 audit findings, all subsequently fixed.

2.1 P1 and P2 findings

Below the fatal tier, the audit catalogued usability and maintainability debt. The highest-value P1 items were the unwired intelligence inputs on the daily feed plan card (all eight context signals defaulted to null or false, so no environmental adjustment ever fired) and the roughly 230 user-visible strings hard-coded in English across feature pages in an application that advertises a Hindi language toggle. P2 items included the copy-pasted dialog code noted above, static break-even figures presented as live analytics in the finance module, and mock data tiles in the tray-check UI that rendered identical values for every pond.

> **Priority model applied.** P0 (fatal: data loss, crashes, fabricated data) → P1 (critical workflows and usability) → P2 (structural refactors and consistency) → P3 (polish). Implementation proceeded strictly in that order; no P2 refactor was started until every P0 carried a verified fix and a green test run.

3. UI/UX Improvements

User-facing corrections concentrated on three themes: honesty of feedback, correctness of labels against engine reality, and reachability of existing screens. The tray-check control group now mirrors the check-tray engine's four actual zones—clean, normal, moderate, surplus—with representative percentages inside each band, replacing a three-button set whose “Normal” label actually submitted a value deep inside the engine's moderate zone and offered no moderate option at all. The buttons also wrap responsively instead of overflowing on narrow devices.

Zone	Old buttons	New buttons	Engine threshold
Clean	Clean (0%)	Clean (0%)	leftover < 1%
Normal	—	Normal (1–10%)	1% ≤ leftover ≤ 10%
Moderate	Normal (25%) — mislabelled	Moderate (11–25%)	10% < leftover ≤ 25%
Surplus	Surplus (50%+)	Surplus (> 25%)	leftover > 25%

Table 3 — Tray-check labels before and after alignment with the engine.

Export interactions now tell the truth. The previous flow showed a success snackbar for a CSV that was never written; the new service writes a timestamped file, opens the operating system share sheet so the file can actually travel to WhatsApp, e-mail, or cloud drive, and falls back to an explicit “saved to path” message where sharing is unavailable. An empty-state guard refuses to export header-only files instead of claiming success, and record counts are echoed back so the farmer can sanity-check the output.

Global search gained real navigation targets: inventory results now route to the previously unreachable feed, medicine, and fuel detail pages, resurrecting 855 lines of orphaned UI. Search result tiles also carry type badges and richer subtitles so items with similar names remain distinguishable at a glance.

4. Layout & Workflow Improvements

The largest structural cleanup removed roughly 500 lines of dialog code duplicated between the Manage page and the pond-scoped batch flow. The two copies had already diverged: one computed the next batch number per pond while the other hard-coded 1, and the harvest copies disagreed on which fields were mandatory. Both pages now delegate to a single shared module (*batch_action_dialogs.dart*, 803 lines including documentation) that owns stocking, hapa survival checks, and harvest completion. When invoked from a pond-scoped card the dialog locks the target pond; from the global Manage page it offers an active-pond picker.

Site editing became crash-proof against historical data. The water-source dropdown previously enumerated a fixed value set that differed between the creation and edit flows, so editing a site created by the other flow threw an assertion. A new *WaterSourceCatalog* merges both the modern and legacy value sets, renders any stored value—known or legacy—with a human label, and only offers catalogued values for new selections. The pattern is deliberately defensive: databases restored from older backups can never wedge the edit screen again.

Feed logging received a focused pass: the multi-pellet ration breakdown persists per-feed selections across slot changes, and the edit-mode title no longer falls back to a missing translation key. Water-quality history gained a dedicated bottom sheet with parameter-trend chips, letting operators review a pond's recent DO/ammonia/pH trajectory without leaving the entry screen.

5. Functional & Data-Integrity Improvements

The harvest completion flow was rewritten around derivation from real records. Estimated shrimp count is computed as harvest weight converted to grams divided by the latest sampled ABW (or the stocking target weight when no sample exists); actual FCR divides the batch's total consumed feed by the harvest weight; survival compares the derived count against stocked count, clamped to a physical 0–100% range, with the latest hapa-check survival as a fallback signal. Every figure displayed in the confirmation dialog is now computed, labelled as an estimate where it is one, and stored with the harvest record.

23/23	0	8
Tables round-tripped by backup v2	Fabricated constants in harvest flow	Intelligence signals wired to feed plan

5.1 Backup and restore, version 2

BackupService v2 exports and restores all 23 user-data tables with their full column sets—including fields that v1 silently dropped such as water source, hapa stocked count, target DOC, notes, and the dynamic feed-items JSON. Restore is round-trip-safe and backward-compatible: v1 backup files remain restorable, with missing keys treated as empty and missing columns falling back to schema defaults. The destructive wipe that precedes a restore now calls a single *wipeAllUserData()* routine—the same one used by factory reset—inside one transaction, so the wipe list can never again drift between features or execute partially.

Aspect	Backup v1 (before)	Backup v2 (after)
Tables exported	10	23 (complete schema)
Tables restored	5 of 10 exported	All 23, full columns
Tables wiped on restore	15 (5 never backed up)	Same atomic list as factory reset
v1 file compatibility	—	Restores with schema-default fallbacks
Wipe atomicity	Sequential deletes	Single transaction, all-or-nothing

Table 4 — Backup coverage before and after the rewrite.

5.2 Real CSV exports and wired intelligence

Reports and inventory export buttons now route through a shared CsvExportService that serialises the displayed rows, writes a timestamped file under the application's documents directory, and hands it to the platform share sheet. The daily feed plan card finally receives its environment: tray leftover percentage, latest dissolved oxygen, latest ammonia, rain risk, heat risk, and mortality spike flags all flow from live provider data into the plan engine, so its feed-reduction advice now reacts to actual pond conditions instead of silently doing nothing.

6. Design System

AquaLog's design system is a pragmatic Material 3 implementation with a growing library of shared *aqua_** widgets in *core/widgets/*. The audit found the system healthy at its core—consistent spacing, a coherent blue-green palette, and typed status chips—but suffering from bypass: high-stakes dialogs were hand-rolled twice instead of shared, and value-set knowledge (water sources) was inlined into pages instead of centralised. This engagement extended the system along its existing grain rather than replacing it.

Module	Role	Status
aqua_scaffold.dart	Page shell with consistent app bar, background, offline banner	Pre-existing, unchanged
aqua_metric_card.dart	KPI tile with trend strip used across dashboards	Pre-existing, unchanged
aqua_empty_state.dart	Illustrated empty states for first-run screens	Pre-existing, unchanged
aqua_status_chip.dart	Typed status pills (active, harvested, low-stock)	Pre-existing, unchanged
aqua_search_dialog.dart	Global search with type routing	Extended: inventory detail routing, richer result tiles
batch_action_dialogs.dart	Single source of truth for stocking / hapa / harvest	New (803 lines), replaces ~500 duplicated lines
config/water_sources.dart	Merged value catalog + labels for water sources	New, defensive against legacy DB values
csv_export_service.dart	Real CSV write + share sheet + honest fallbacks	New (136 lines)
water_quality_history_sheet.dart	Parameter-trend bottom sheet	New (142 lines)

Table 5 — Shared design-system modules after this engagement.

Two conventions were reinforced for future contributors. First, dialogs that mutate batch lifecycle state belong in the shared module, not in page files; the pond-scoped and global entry points differ only in the *fixedPond* parameter they pass. Second, enumerated domain values live in a catalog under *core/config/* with a label map and an *isKnown* guard, so UI dropdowns can render legacy persisted values instead of crashing on them. Both conventions are documented in the module headers.

7. Technical & Code-Quality Improvements

Every change in this engagement was gated on two verifiable commands: *flutter analyze* and *flutter test*. The implementation landed in two waves; the first wave was written before a Flutter SDK was available in the working environment and therefore shipped unverified, which this engagement's first task was to rectify. After installing the SDK (3.47.3, matching the project's Dart 3.13 constraint), the initial analyzer run surfaced fifteen issues, including five hard errors where four files imported *share_plus* without the dependency ever having been declared in pubspec.yaml. The dependency was added via *flutter pub add*, four unused or duplicate imports were removed, and one doc comment was escaped for the HTML linter.

0	164/164	15 → 0
Analyzer issues after fixes	Tests passing, 21 suites	Analyzer findings resolved

The test suite proved its worth immediately: it exercised the rewritten backup service (round-trip and v1-compatibility cases), the batch hierarchy invariants, the calculation engines, and named regression suites for earlier defect classes. All 164 cases pass against the modified tree, giving high confidence that the P0 fixes did not disturb neighbouring behaviour. Import hygiene was tightened in the three files that had carried dead imports since the dialog deduplication, and the *wipeAllUserData()* helper was placed on the database class itself so both factory reset and restore consume one authoritative list.

One tooling improvement was persisted for future sessions: the Flutter SDK installation script now targets a user-writable directory, since container sandboxes frequently deny writes to /opt. The script lives alongside the project and documents the exact SDK version constraint, preventing the version drift that contributed to the unverified first wave.

8. Changed Files

The two implementation commits touched 19 files in the application tree with a net addition of roughly 2,600 lines, of which the largest single additions are the backup service rewrite and the shared dialog module. The manifest below lists every file with its change size; deletions in the Manage and Pond Creation pages correspond line-for-line to the new shared module that absorbed them.

File	Change	Summary
core/services/backup_service.dart	+1,124	v2 rewrite: all 23 tables, round-trip safe, v1 compatible

File	Change	Summary
manage/widgets/batch_action_dialogs.dart	new (803)	Shared stocking / hapa / harvest dialogs with real derivations
core/services/backup tests (test/backup_restore_test.dart)	covered	Round-trip and v1-compatibility cases pass
core/providers/batch_dashboard_providers.dart	+59	Intelligence signals (tray, DO, ammonia, weather, mortality)
core/widgets/aqua_search_dialog.dart	+236	Inventory detail routing, richer result tiles
features/water_quality/water_quality_history_sheet.dart	new (142)	Parameter-trend bottom sheet
core/services/csv_export_service.dart	new (136)	Real CSV write, share sheet, honest fallbacks
features/manage/manage_page.dart	−527 net	Duplicated dialogs replaced by delegates to shared module
features/manage/pond_creation_page.dart	−445 net	Same deduplication on the pond-scoped flow
features/settings/settings_page.dart	+68	Backup/restore UI wired to v2 service, share intents
core/database/app_database.dart	+42	wipeAllUserData(): atomic, shared with factory reset
core/config/water_sources.dart	new (35)	Merged catalog, labels, legacy-value guard
features/feed/feed_logging_page.dart	+15	Tray zone buttons aligned to engine thresholds
features/dashboard/batch_dashboard_view.dart	+54	DailyFeedPlanCard receives all intelligence inputs
features/reports/reports_page.dart	+18	Real CSV export on report tables
features/inventory/inventory_reports_page.dart	+14	Real CSV export on inventory tables
features/water_quality/water_quality_entry_page.dart	+8	History sheet entry point
pubspec.yaml / pubspec.lock	+7 deps	share_plus added via flutter pub add
3 feature files	imports	Unused/duplicate imports removed after analyzer pass

Table 6 — Complete changed-file manifest.

9. Remaining Issues

The engagement closed with every P0 fixed and verified, but honest accounting requires listing what remains open. The dominant item is internationalisation: the application ships a Hindi language toggle, yet roughly 750 widget-level string literals across feature pages remain hard-coded in English—including substantial portions of the new shared dialogs written during this engagement to match the surrounding code's convention. Until the string catalog grows to cover these, the Hindi setting delivers a partially translated experience.

- **i18n coverage gap** — ~750 hard-coded English strings vs. a catalog of ~100 keys; the Hindi toggle is only partially effective.
- **Backup confidentiality** — exports are plain JSON with no passphrase encryption; a stolen backup file exposes the full farm ledger.
- **Static finance analytics** — the break-even card still presents a fixed benchmark rather than computing from the batch's own expense ledger.
- **Dependency age** — 23 packages have newer versions blocked by current constraints (riverpod 2.6, fl_chart 0.70); upgrades deserve a dedicated pass with the test suite as the gate.
- **No CI pipeline** — analyze + tests run manually; nothing prevents an unverified commit from regressing the P0 classes fixed here.
- **Reference-only folders excluded** — the uploaded archive carried an old web app and design-stitch specs; they were removed from the working copy and should be tracked separately if still needed.

None of these items regress previously working behaviour, and none blocks release. They are sequenced recommendations, ranked by user-visible impact per unit of effort.

10. Future Recommendations

The roadmap below sequences the remaining work into four increments, each independently shippable and gated on the same two commands used throughout this engagement. The guiding principle remains improve-over-rebuild: extend the shared modules and catalogs introduced here rather than adding parallel mechanisms.

Phase	Scope	Effort	Gate
1 — Trust hardening	Passphrase-encrypted backups (AES over the JSON payload); break-even card computed from the live expense ledger	2–3 days	Backup round-trip tests extended to ciphertext
2 — Full bilingual	Extract all user-visible strings to AppStrings with EN/HI pairs; lint rule banning hard-coded Text() literals	1–2 weeks	i18n coverage script reports 100% of screens
3 — Continuous verification	CI running flutter analyze + flutter test on every commit; coverage report on the P0 regression suites	1 day	Pipeline blocks a deliberately broken commit
4 — Platform refresh	Upgrade riverpod / fl_chart / drift within constraints; widget tests for the shared dialog module	1 week	Full suite green on upgraded tree

Table 7 — Recommended roadmap, ordered by impact per effort.

Two further investments would compound over time. A golden-test suite for the tray zone buttons and harvest dialog would lock the label-to-engine alignment this engagement established, catching future threshold changes at review time. And because the intelligence layer now actually receives environmental signals, its advice quality is ready for field validation: instrumenting which feed-plan adjustments operators accept would turn AquaLog's own usage into the next training signal for its recommendations.

> **Delivery state.** Working tree: *flutter analyze* clean, 164/164 tests passing, three git commits ahead of the uploaded baseline. The improved source and this report are delivered together so every claim above can be re-verified from the tree it describes.